Operacijski sustavi (OS - 199570)



Nositelj: doc. dr. sc. Ivan Lorencin
Asistent: Luka Blašković, mag. inf.

Ustanova: Sveučilište Jurja Dobrile u Puli, Fakultet informatike u Puli

Fakultet informatike u Puli

(1) Uvod u operacijske sustave

#1 OS

Svaki računalni sustav sastavljen je od procesora, radnog spremnika, vanjskih spremnika i različitih ulazno-izlaznih naprava. Te komponente čine sklopovlje računala. Međutim, samo sklopovlje nije nam od velike koristi ako uz njega ne postoji odgovarajuća programska oprema. Različiti primjenski programi, s pomoću kojih korisnici računala obavljaju sebi korisne zadatke, transformiraju računalni sustav u koristan virtualni stroj. Kao potpora svim tim raznovrsnim programima postoji skup osnovnih programa koji omogućuju prevođenje radnih zahvata na računalu. Taj se skup osnovnih programa naziva operacijskim sustavom. Na ovom kolegiju studenti će naučiti upravljati operacijskim sustavom računala kroz naredbeni redak koji za razliku od grafičkog sučelja (gdje se koriste miš i vizualni elementi), koristi isključivo tekstualne naredbe, dakle puno tipkovnice. Dobro naredbenog retka ključni je *skill* je za svakog informatičara, jer je de facto standard u administraciji sustava, automatizaciji zadataka, radu na udaljenim poslužiteljima te modernom *developmentu*. Moderna softverska rješenja gotovo uvijek se implementiraju u oblaku, gdje su alati kroz naredbeni redak neizostavni za upravljanje resursima, implementaciju i održavanje aplikacija. Stoga, iako se na prvi pogled može činiti da je naredbeni redak "stari" alat, on je zapravo ključan za suvremeni informatički rad i razvoj. Na ovom kolegiju studenti će steći praktične vještine rada s naredbenim retkom, što će im omogućiti učinkovito upravljanje operacijskim sustavom i pripremiti ih za izazove modernog IT okruženja. *Let's get started!*

🆙 **Posljednje ažurirano: 12.9.2026.**

Sadržaj

- Operacijski sustavi (OS - 199570)
- (1) Uvod u operacijske sustave
  - Sadržaj
- 1. Uvod
- 2. Naredbeni redak (CLI)
  - 2.1 Bash - Bourne Again Shell
  - 2.2 Kako pokrenuti bash?
    - Unix-like OS
    - Windows OS
- 3. Osnovne bash naredbe
  - Naredba `pwd`
  - Naredba `ls`
  - Naredba `cd`
  - Naredba `clear`
  - Naredba `echo`
- 4. Bash naredbe za manipulaciju datotekama i direktorijima
  - Naredba `mkdir`
  - Naredba `rmdir`
  - Naredba `touch`
  - Naredba `cp`
  - Naredba `mv`
  - Naredba `rm`
- 5. Tablica osnovnih bash naredbi
- Zadaci za Vježbu 1

1. Uvod

Operacijski sustav (*eng. Operating System*, skraćeno *OS*) je temeljni softver svakog računalnog sustava. Njegova primarna funkcija je upravljanje hardverskim resursima te osiguravanje učinkovitog i pouzdanog sučelja između korisnika i hardverskih komponenti, omogućujući tako učinkovit i stabilan rad računala.

Dva važna zadatka operacijskog sustava su:

1. Olakšavanje uporabe računala korisnicima
2. Djelotvorno iskorištavanje svih dijelova računala

Naime, unutar računala može se istodobno odvijati više poslova.

Primjer: istodobno dok se procesor "brine" o izvođenju jednog niza instrukcija, pristupni sklop pisača može iz glavnog spremnika prenositi sadržaj na pisač (printer). OS se mora pobrinuti da se procesor "prebacuje" s izvođenja jednog niza instrukcija na drugi i podržati **višezadaćnost**.

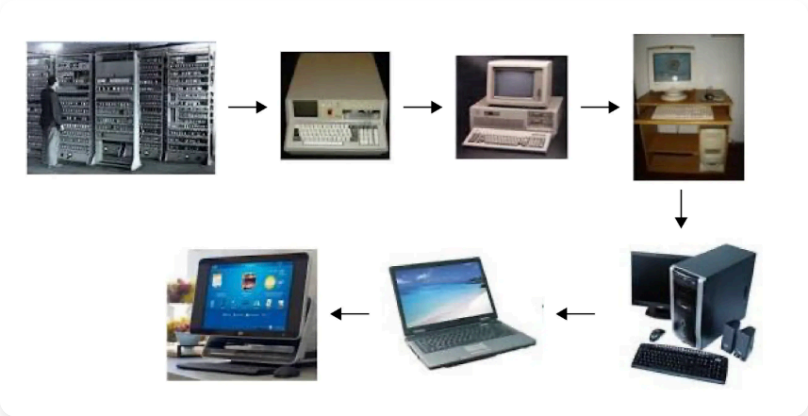
Rani razvoj OS-a bio je usmjeren prema sustavima razvijenim za tzv. *mainframe* računala, s naglaskom na serijsko izvršavanje zadatka i minimizaciju potrebe za interakcijom s korisnikom. Samim tim, operacijskih sustava bilo je mnogo i bili su vrlo specifični za određeni računalni sustav, odnosno pojedino *mainframe* računalo. Tipično, svaki put kad bi proizvođač računala izbacio novi model, morao bi razviti i novi operacijski sustav za taj model, a samim tim i prilagoditi sve programe.

Uvođenjem **višezadaćnosti** (eng. *multitasking*) i **višekorisničkog** (*eng. multiuser workload*) rada početkom 70-ih godina 20. stoljeća, operacijski sustavi su postali sposobni istovremeno izvršavati više zadataka, što je danas normalno u modernom računarstvu.

Definicija operacijskog sustava značajno je evoluirala. Naime, nekad su operacijski sustavi predstavljali isključivo osnovni skup programa nužnih za funkcioniranje računala, dok su danas postali kompleksni sustavi koji uključuju niz dodatnih programa i alata.

Možemo to usporediti s automobilima – rani modeli nisu imali brzinomjer, radio ili klimu, ali danas su ti dodaci standardni i nezamislivo je kupiti novi automobil bez njih. Slično tome, moderni operacijski sustavi više nisu samo temeljni alati za upravljanje računalom, već dolaze s nizom dodatnih programa, poput grafičkog sučelja, web preglednika, uređivača teksta, datotečnog sustava pa i razno-raznih *third party* aplikacija.

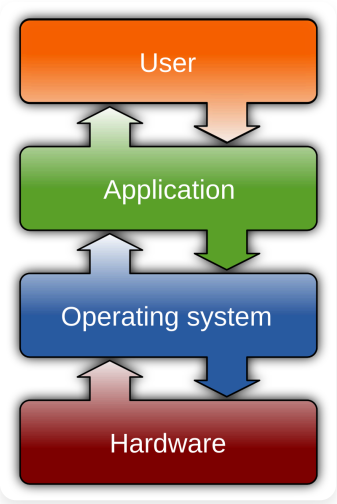
Zbog toga se operacijski sustav danas često doživljava kao "**platforma**" (ne PaaS!) – cjeloviti korisnički sustav koji uključuje integrirano sučelje, datotečni sustav, uređivače teksta, tablica, slika i druge korisne alate. Više nije riječ samo o skupu osnovnih programa, već o ukupnom okruženju u kojem korisnici mogu obavljati razne zadatke na računalu.



> Slika 1. Evolucija računala, od *mainframe* računala (gore lijevo) do modernih prijenosnih računala i *all-in-one* računala (dolje sredina, lijevo)

Svaki računalni sustav moguće je prikazati kroz hijerarhijsku strukturu koja se sastoji od četiri razine (gdje je: 1 - najapstraktnija, 4 - najkonkretnija):

1. **Korisnička razina** (*eng. User Layer*) - najviša razina na kojoj se nalaze programi koje koriste krajnji korisnici
2. **Razina primjenskih programa** (*eng. Application Layer*) - razina na kojoj se nalaze programi koji koriste usluge operacijskog sustava
3. **Operacijski sustav** (*eng. Operating system*) - posrednik između fizičkih komponenti računala i ostalog softvera
4. **Razina sklopovlja** (*eng. Hardware Layer*) - najniža razina na kojoj se nalaze fizičke komponente računala



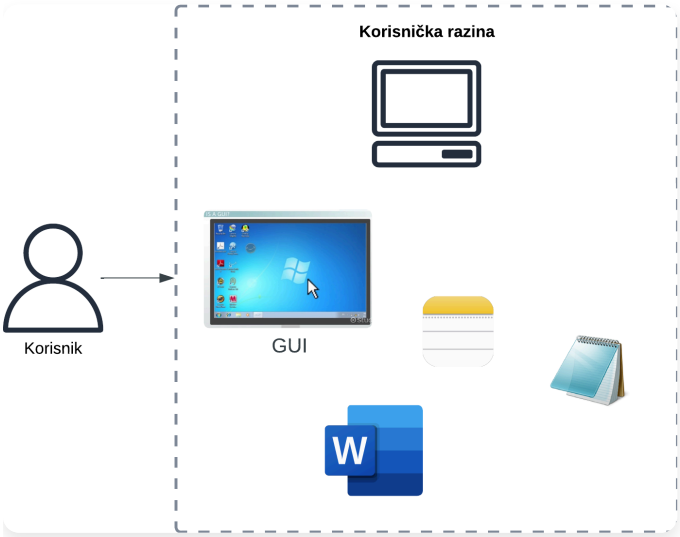
> Slika 2. Četiri razine računalnog sustava i njihova međusobna povezanost/komunikacija

Pokazat ćemo primjer kako ovo izgleda u praksi na primjeru otvaranja tekstualne datoteke.

1. Korisnička razina

Na najvišoj razini, korisnik pokreće program za obradu teksta (npr. Microsoft Word, VS Code, Mozilla Firefox, File Explorer). Ovdje se odvija **interakcija s grafičkim korisničkim sučeljem** (*eng. GUI*) na način da korisnik klikne na ikonu programa ili otvori datoteku iz nekog direktorija.

Aplikacija na ovoj razini ne zna kako se podaci pohranjuju na disku niti kako pristupiti disku, ali zna kako prikazati podatke korisniku jednom kad ih dobije.

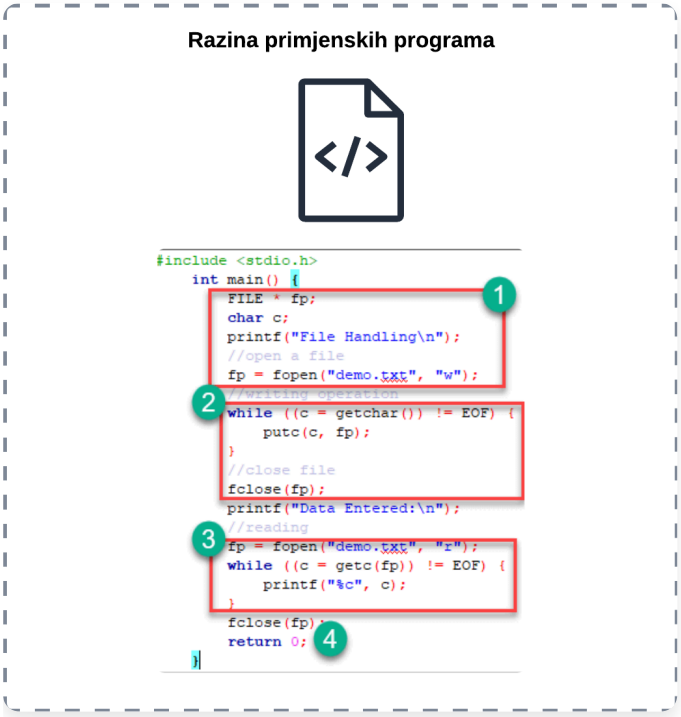


> Slika 3. Korisnička razina: korisnik pokreće program za obradu teksta kroz GUI

2. Razina primjenskih programa

Nakon što korisnik pokrene tekstualni uređivač na korisničkom sloju, program koristi niz **aplikacijskih programskih sučelja** (*eng. Application programming interface*, skraćeno *API*) za komunikaciju s operacijskim sustavom. Recimo: `C` jezik nudi funkciju `fopen()` za čitanje datoteke, u Pythonu bi to bila funkcija `open()`, u JavaScriptu `fs.readFile()`, itd.

U ovom kontekstu, zadaća primjenskih programa je da **koriste sučelje koje nudi operacijski sustav** kako bi pristupili datotekama, memoriji, mreži i ostalim resursima i osiguraju siguran pristup i učinkovit rad s tim resursima. Na neki način, oni **apstrahiraju složenost pristupa resursima i omogućuju programerima da se fokusiraju na rješavanje problema**.

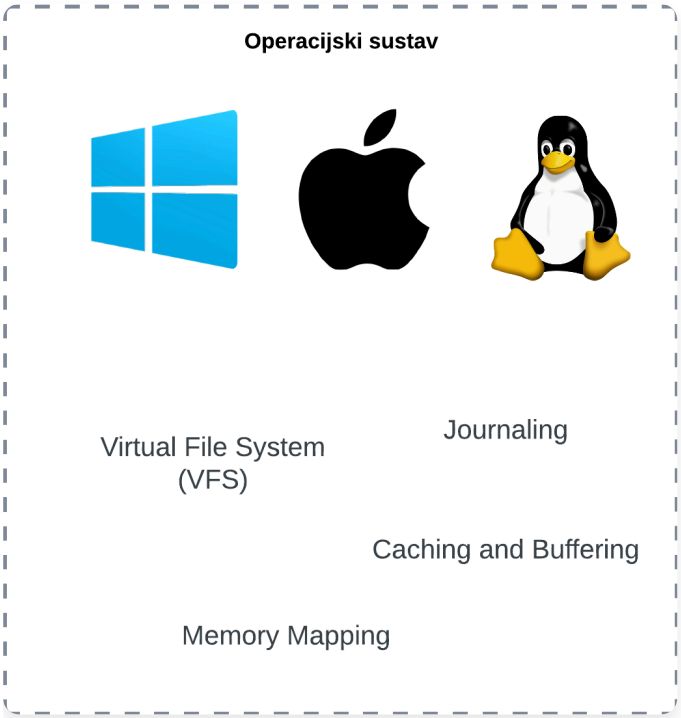


> Slika 4. Razina primjenskih programa: primjenski program koristi funkciju poput `fopen()` za pristup datoteci preko sučelja sustava

3. Operacijski sustav

Operacijski sustav prima zahtjev preko API-ja i obavlja niz operacija kako bi zadovoljio zahtjev primjenskog programa. Na primjer, ako je primjenski program zatražio otvaranje tekstualne datoteke, OS mora odraditi sljedeće operacije (redoslijed operacija u ovom slučaju nije nužno ovakav):

- **učitati datoteku**: OS mora pronaći odgovarajuću datoteku na disku
- **provjeriti dozvole pristupa**: OS mora provjeriti dozvole pristupa za odgovarajuću datoteku
- **upravljati radnom memorijom**: OS mora osigurati dovoljno radne memorije za učitavanje datoteke
- **rukovati potencijalnim greškama i iznimkama**: OS mora obraditi greške koje se mogu pojaviti tijekom operacije
- **vratiti potrebne informacije** natrag prema primjenskom programu
- **osloboditi resurse** nakon završetka operacije



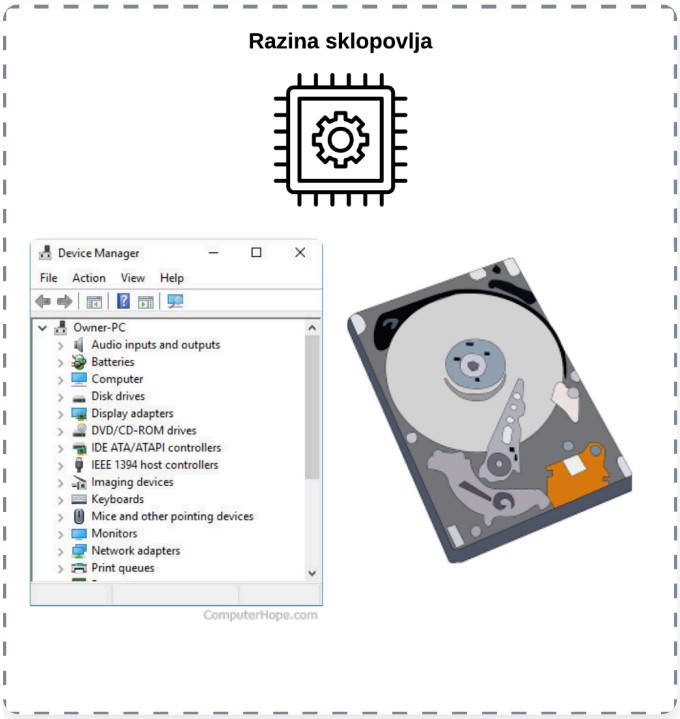
> Slika 5. Operacijski sustav: Upravljanje resursima i povratna komunikacija s primjenskim programima

4. Razina sklopovlja

Na najnižoj razini, operacijski sustav koristi upravljačke programe (*eng. Device drivers*) kako bi komunicirao s fizičkim komponentama računala.

Upravljački programi su posebni programi koji omogućuju operacijskom sustavu da komunicira s hardverom, kao što su diskovi, memorija, grafičke kartice, mrežne kartice itd. Upravljačke programe u pravilu razvijaju proizvođači hardvera, a možemo ih zamisliti kao posrednike između operacijskog sustava i *firmware* softvera koji je programiran u samom hardveru.

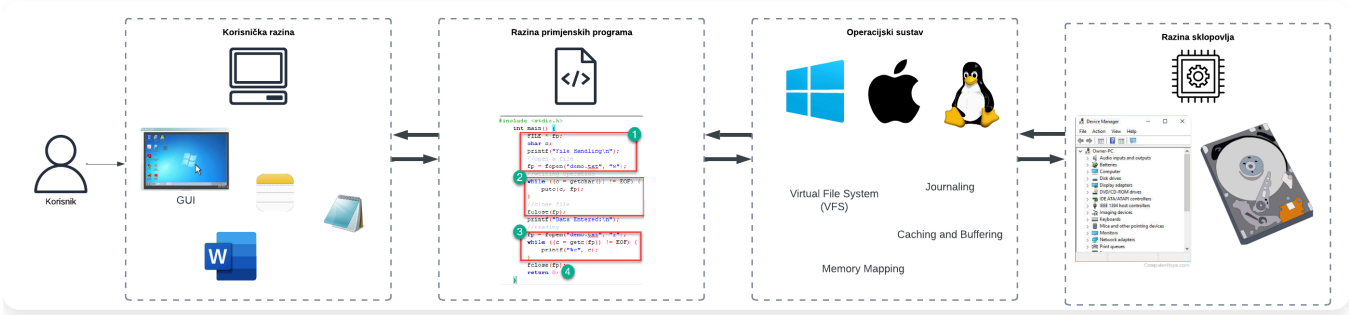
Bez odgovarajućeg upravljačkog programa, operacijski sustav ne može ispravno koristiti povezani hardver. Ovo je najkonkretnija razina na kojoj se odvija komunikacija s fizičkim komponentama računala.



> Slika 6. Razina sklopovlja: upravljački programi omogućuju komunikaciju s fizičkim komponentama računala

Naravno, kao što je i vidljivo na ilustraciji iznad, komunikacija između ovih razina je **dvosmjerna** (↔).

U nastavku je prikazana **ukupna komunikacija između različitih razina računalnog sustava** na opisanom primjeru otvaranja tekstualne datoteke:



> Slika 7. Ukupni prikaz dvosmjerne komunikacije između različitih razina računalnog sustava na opisanom primjeru otvaranja tekstualne datoteke

TL;DR

- **Korisnička razina**: korisnik pokreće program za obradu teksta
- **Razina primjenskih programa**: primjenski program koristi "API operacijskog sustava" za pristup resursima, odnosno standardnu biblioteku i sistemske pozive
- **Operacijski sustav**: OS obavlja operacije koje zadovoljavaju zahtjev primjenskog programa
- **Razina sklopovlja**: upravljački programi omogućuju komunikaciju s fizičkim komponentama računala u svrhu izvršavanja stvarne operacije

Pitanje za razmišljanje:

Nakon ovog kratkog uvoda u operacijske sustave, na sljedećih nekoliko vježbi ćemo se ustvari baviti učenjem **sučelja naredbenog retka** (eng. *Command Line Interface*, skraćeno *CLI*).

U koji od navedenih dijelova računalnog sustava uopće spada naredbeni redak? Argumentirajte odgovor.

▼ **Spoiler alert! Odgovor na pitanje**

Naredbeni redak možemo svrstati u korisničku razinu računalnog sustava. Premda nam omogućuje pristup resursima operacijskog sustava i interakciju s njime, **naredbeni redak je prije svega sučelje koje koristi krajnji korisnik** za upravljanje računalom.

Štoviše, kada koristimo naredbeni redak, u pozadini često rade brojni primjenski programi (kao i u slučaju GUI-a) koji nam omogućuju pristup operacijskom sustavu, a često nismo ni svjesni njihove prisutnosti.

2. Naredbeni redak (CLI)

Naredbeni redak (*eng. Command Line Interface*, skraćeno *CLI*) je tekstualno sučelje koje omogućuje korisnicima unos i izvršavanje naredbi u operacijskom sustavu. Za razliku od grafičkog korisničkog sučelja (*eng. Graphical User Interface*, skraćeno *GUI*), koje koristi miš i vizualne elemente za interakciju s korisnikom, **naredbeni redak koristi isključivo tekstualne naredbe**.

Prvi interaktivni naredbeni redci pojavili su se još 60-ih godina prošlog stoljeća, a sredinom 70-ih i početkom 80-ih godina počinju se **standardizirati**.

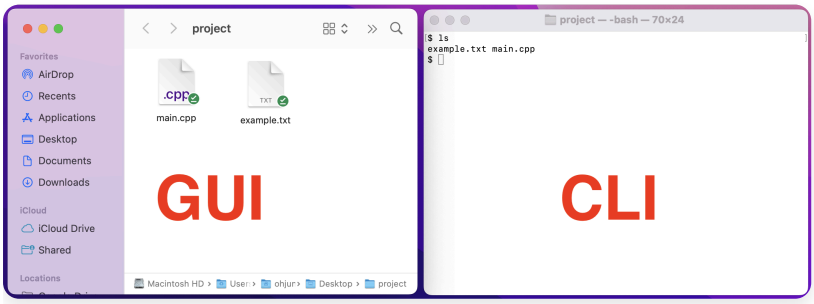
Premda se većina današnjih računalnih sustava oslanja na GUI umjesto na CLI, mnogi aplikacijski i pomoćni programi nemaju korespondirajući GUI, već se koriste isključivo kroz CLI.

CLI ima mnogo sinonima i manjih varijacija, međutim **za početak je dovoljno znati da se radi o tekstualnom sučelju koje se bazira na unosu naredbi i prikazu rezultata izvršavanja tih naredbi**.

Za CLI se često koriste i sljedeći "sinonimi" (premda značenje nije potpuno isto - više o tome u nastavku):

- Terminal
- Shell
- Console
- Command prompt
- Command line
- PowerShell
- Interpreter
- "Ono crno"
- i mnogi drugi...

> U nastavku više o razlikama između ovih pojmova.



> Slika 8. GUI vs CLI

Kao generalni pojam, u ovim skriptama najčešće će se upotrebljavati termin **naredbeni redak**, odnosno skraćena verzija: *CLI*.

2.1 Bash - Bourne Again Shell

Rekli smo da se termini CLI, terminal i *shell* često upotrebljavaju kao sinonimi, različiti nazivi za istu stvar. Međutim, ako malo zagrebemo ispod površine, možemo definirati neke razlike između ovih pojmova:

Terminal je **program** koji pruža korisniku tekstualno sučelje/okruženje za interakciju s računalom.

- dopušta korisniku unos naredbi i prikazuje rezultate izvršavanja tih naredbi
- možemo ga zamisliti kao **omotač** (*eng. wrapper*) oko *shella* koji nam omogućuje da unosimo naredbe
- ne obrađuje naredbe, već služi kao posrednik (medij)

Moderni terminali su: `GNOME Terminal`, `Konsole`, `iTerm2`, `Alacritty`, `Windows Terminal`...

📌**Analogija**: Terminal je alat, poput ekrana i tipkovnice, koji koristimo za komunikaciju s računalom.

CLI (*hrv. Naredbeni redak*) je generalni **koncept** koji se odnosi na bilo koje tekstualno sučelje koje omogućuje korisnicima interakciju s računalom putem naredbi.

- CLI radi unutar terminala
- **nije program, već način interakcije s računalom** (kao što je i GUI način interakcije)
- mnogi programi imaju svoje vlastito CLI sučelje, npr. `git`, `npm`, `docker`, `python`, `aws` ...

📌**Analogija**: CLI je poput jezika (npr. Engleski, Hrvatski) koji koristimo za svakodnevni razgovor. To je metoda interakcije.

Shell (*hrv. ljuska*) se odnosi na **specifični program** koji interpretira naredbe koje korisnik unosi unutar terminala.

- služi kao **posrednik između korisnika i operacijskog sustava**
- **omogućava interakciju s OS-om**
- može biti **interaktivan** (unosom tekstualnih naredbi) ili **skriptni** (izvršavanje napisanih *shell* skripti - *#soon*)

Primjeri poznatih *shellova*:

- **Bourne shell** (*sh*) - jedna od prvih i najpoznatijih *shell* implementacija (70-ih godina)
- **Bash** (*Bourne Again Shell*) - najpopularniji *shell* za Unix i *Unix-like* sustave
- **Zsh** (*Z shell*) - moderna alternativa bashu
- **Fish** (*Friendly Interactive Shell*) - još jedna moderna alternativa bashu
- **tcsh** (*Tenex C Shell*) - starija alternativa bashu
- **CMD** (*Command Prompt*) - stari *shell* za Windows OS
- **PowerShell** - novi *shell* za Windows OS, ali dostupan i na drugim platformama
- i mnogi drugi...

📌**Analogija**: *shell* je poput prevoditelja koji interpretira/prevodi naš jezik i sudjeluje u konverzaciji.

Jedno "kviz pitanje":

U gornjim opisima *shellova* spominje se pojam Unix i *Unix-like* sustavi. Što je to Unix i što znači pojam *Unix-like* sustavi?

▼ **Spoiler alert! Odgovor na pitanje**

Unix je operacijski sustav koji je razvijan u Bell Labsu sredinom 60-ih i početkom 70-ih godina. Unix je bio jedan od prvih operacijskih sustava koji je podržavao višekorisnički i višezadaćni rad, a njegova arhitektura i dizajn bili su temelj za mnoge moderne operacijske sustave.

Unix je bio zatvorenog koda, ali je kasnije razvijena otvorena verzija pod nazivom BSD (Berkeley Software Distribution) koja je postala temelj za mnoge *Unix-like* operacijske sustave.

Unix-like sustavi su operacijski sustavi koji su dizajnirani na temelju Unixa, ali nisu nužno kompatibilni s njim. To znači da su *Unix-like* sustavi inspirirani Unixom, ali su razvijeni odvojeno i ne dijele kod s Unixom.

Najpoznatiji *Unix-like* sustavi su **Linux** i **macOS**

Za lakše razumijevanje, spomenute razlike sažete su u tablici:

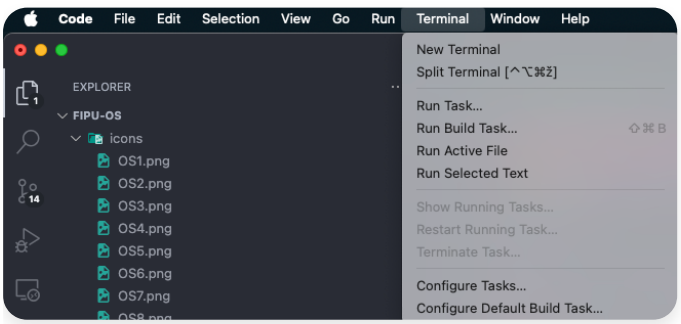
Ključne razlike između terminala, CLI-ja i *shella*

Koncept	Objašnjenje	Primjeri
Terminal	Program koji pruža tekstualno sučelje, ali ne obrađuje naredbe.	`Windows Terminal`, `Command Prompt`, `GNOME Terminal`, `Konsole`, `iTerm2`, `Warp`, `Alacritty`
CLI (*hrv. Naredbeni redak*)	Način interakcije sa sustavom kroz naredbe, suprotnost je GUI.	`Git CLI`, `Docker CLI`, `Linux CLI`, `AWS CLI`, `Netlify CLI`
Shell (*hrv. ljuska*)	Program koji interpretira naredbe i vrši interakciju s operacijskim sustavom.	`bash`, `zsh`, `fish`, `PowerShell`

> **Napomena**: neke programe možemo interpretirati i kao CLI i kao *shell*, ali i kao terminale. Primjer takvog programa je `Command Prompt (cmd.exe)` u Windows sustavu. On je prvenstveno *shell* jer interpretira naredbe, a kao sučelje može koristiti `Windows Terminal`, standalone (vlastiti terminal), `PowerShell` pa i `VS Code terminal`. Isto vrijedi i za mnoge druge *shellove*.

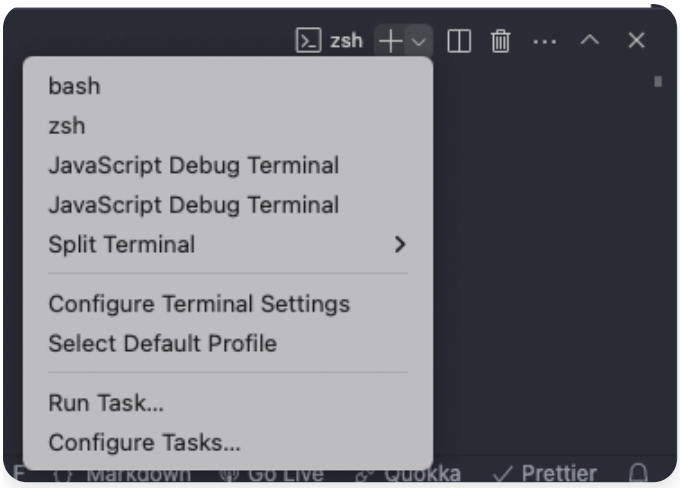
Primjer za lakše razumijevanje ovih razlika:

Ako otvorite VS Code okruženje, možete pokrenuti novi **Terminal**.



> Slika 9. VS Code → New Terminal

Unutar tog terminala možete pokrenuti ***shell*** (npr. bash) i koristiti ga kao **CLI** za izvršavanje naredbi.



> Slika 10. Terminal → bash

Međutim, uočite i opcije poput: `JavaScript Debug Terminal`, `Python Debug Console`. Ovo su također terminali, ali su specifični za određene programske jezike i koriste se za **debugiranje izvršivog koda**, **a ne kao CLI** za interakciju s OS-om ili nekim drugim programom.

Zaključno: Unutar istog terminala možemo koristiti različite *shellove* i CLI-eve, ovisno o potrebama i preferencijama.

Mi ćemo na vježbama učiti **bash** (*Bourne Again Shell*), najpopularniji *shell* za Unix i *Unix-like* sustave.



> Slika 11. bash - Bourne Again Shell, najpopularniji *shell* za Unix i *Unix-like* sustave

2.2 Kako pokrenuti bash?

Bash (*Bourne Again Shell*) je popularni Unix *shell* za Unix i *Unix-like* sustave. Razvio ga je **Brian Fox** 1989. godine kao dio GNU projekta, s ciljem pružanja poboljšane i slobodne alternative originalnom Bourne *shellu* (*sh*). GNU projekt, koji je započeo Richard Stallman 1983. godine, razvio je niz alata za operacijski sustav GNU. Iako GNU nije imao vlastiti kernel, kombiniranjem njegovih komponenti s Linux kernelom, koji je 1991. razvio Linus Torvalds, nastao je **GNU/Linux operacijski sustav** (poznat kao Linux).

Bash *shell* dolazi unaprijed instaliran na većini *Unix-like* sustava, **uključujući većinu distribucija Linuxa**. Na macOS-u je bio zadani *shell* do verzije 10.15 (Catalina), nakon čega ga je Apple zamijenio **Z shellom** (*zsh*).

Unix-like OS

Ako koristite *Unix-like* OS, jednostavno otvorite novi terminal i unesite naredbu `bash`:

```
→ bash
```

Ovo će pokrenuti bash *shell* unutar aktivnog terminala.

Primjer ispisa:

```
bash-3.2$
```

Windows OS

Ako koristite Windows OS, postoji više načina za pokretanje bash *shella*.

Svakako se preporučuje korištenje novog Windows Terminala kao sučelja za pokretanje bash *shella*, a ne stari Command Prompt (`cmd.exe`).

Preporučeni način rada je koristeći **Windows Subsystem for Linux** (**WSL**). WSL je Linux okruženje koje se može instalirati na Windows OS-u i omogućuje pokretanje Linux distribucija unutar Windowsa, samim time i korištenje bash *shella*.

> Napomena: WSL je odličan alat za razvojne programere koji rade na Windowsu, ali trebaju Linux okruženje za razvoj, testiranje ili pokretanje svojih aplikacija. Primjerice, kod pokretanja Docker kontejnera (FIPU kolegiji Informacijski sustavi, Raspodijeljeni sustavi, Upravljanje poslovnim procesima) ili rada s AI/DevOps alatima koji su često *Linux-first* u svojoj implementaciji.

Otvorite `Windows Terminal` ili `PowerShell` **kao administrator** i unesite sljedeću naredbu kako biste instalirali WSL:

```
→ wsl --install
```

Ako dobivate grešku, moguće da je već instaliran ili niste pokrenuli terminal kao administrator. U tom slučaju, izlistajte dostupne distribucije i provjerite je li `Ubuntu` instaliran:

```
→ wsl --list --verbose
```

Trebali biste dobiti popis instaliranih Linux distribucija, primjerice:

```
  NAME            STATE           VERSION
* Ubuntu          Stopped         2
```

> Zvjezdica `*` označava zadanu distribuciju

Ako slučajno nemate niti jednu distribuciju, možete ručno instalirati `Ubuntu` distribuciju:

```
→ wsl --install -d Ubuntu
```

Ako ih imate više, postavite zadanu na `Ubuntu`:

```
→ wsl --set-default Ubuntu
```

> Nakon prve instalacije, možda ćete morati ponovno pokrenuti računalo.

Napokon, pokrenite WSL:

```
→ wsl
```

To je to! Kako biste se uvjerili da ste u bash *shellu*, unesite naredbu:

```
→ echo $0
```

Ako se ispiše `bash`, znači da ste u bash *shellu* i možete početi s radom. 😎

3. Osnovne bash naredbe

U ovom dijelu ćemo naučiti nekoliko osnovnih bash naredbi koje će nam pomoći u svakodnevnom radu s naredbenim retkom. Za sada se nećemo baviti složenim naredbama (niti zastavicama).

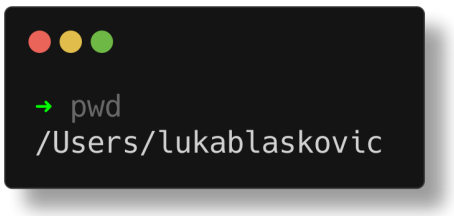
Svaki put kad otvorimo terminal, zapravo se nalazimo u određenom direktoriju na računalu. Direktorij u kojem se trenutno nalazimo naziva se **radni direktorij** (*eng. working directory*).

Prema tome, nije loše započeti upravo od toga, saznati gdje se trenutno nalazimo!

Naredba `pwd`

`pwd` (zapamti kao: *print working directory*) → **ispisuje trenutni radni direktorij kao apsolutnu putanju**

```
→ pwd
```



> Slika 12. Naredba `pwd` ispisuje apsolutnu putanju radnog direktorija

Apsolutna putanja (eng. Absolute Path)

Apsolutna putanja (*eng. Absolute Path*) je putanja koja počinje od korijenskog direktorija (`/` na *Unix-like* sustavima) i vodi do trenutnog direktorija. Na primjer, `/home/<user>/Desktop`.

VAŽNO! Apsolutna putanja je neovisna o trenutnom radnom direktoriju. Drugim riječima, bez obzira na to gdje se nalazimo, apsolutna putanja će uvijek voditi do istog mjesta na disku.

Apsolutna putanja na Windowsu počinje s simbolom diska, npr. `C:\Users\<vaš_username>\Desktop`. Međutim, obzirom da koristimo WSL, dobivamo apsolutnu putanju u Unix stilu s prefiksom `/mnt/` koji označava *mount point* za pristup datotečnom sustavu Windowsa unutar Linux okruženja.

Primjeri apsolutnih putanji:

```
/mnt/c/Users/Marko # Windows (WSL)
/Users/Marko # macOS
/home/Marko # Linux
C:\Users\Marko # Windows
```

- `/mnt/c` je prefiks koji označava `C:\` disk na Windowsu, ali *mountan* kao `/` u WSL-u
- **Važno!** Uočite obrnuti redoslijed separatora (`\` na Windowsu, `/` na *Unix-like* sustavima)!

Relativna putanja (eng. Relative Path)

Relativna putanja (*eng. Relative Path*) je putanja koja se odnosi na trenutni radni direktorij.

Na primjer, ako se nalazimo u direktoriju `/home/<user>`, relativna putanja do direktorija `Desktop` je `Desktop`.

Primjeri relativnih putanji:

```
Desktop # relativna putanja do direktorija Desktop ako se nalazimo u /home/<user> odnosno /mnt/c/Users/<user>
marko/Desktop # relativna putanja do direktorija Desktop ako se nalazimo u /home odnosno /mnt/c/Users
```

Ako se nalazimo u direktoriju na apsolutnoj putanji `/home/sanjasanjic/Desktop` i želimo pristupiti datoteci `file.txt` na radnoj površini, relativna putanja do te datoteke bila bi `file.txt`.

Ako želimo pristupiti nekoj slici koja se nalazi u direktoriju `slike-ljeto-24-25`, relativna putanja do određene slike bila bi `slike-ljeto-24-25/ime-slike.jpg`.

Naredba `ls`

`ls` (zapamti kao: *list*) → **ispisuje sadržaj direktorija (direktoriji + datoteke)**

Sintaksa:

```
→ ls
# ili
→ ls <putanja>
```

- gdje `<putanja>` može biti relativna ili apsolutna
- bez argumenta, `ls` ispisuje sadržaj trenutnog radnog direktorija (onog na koji pokazuje `pwd`)

Primjeri:

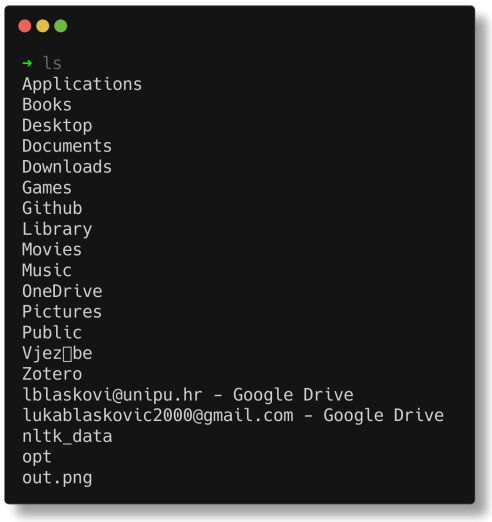
```
→ ls # ispisuje sadržaj trenutnog radnog direktorija

# ili

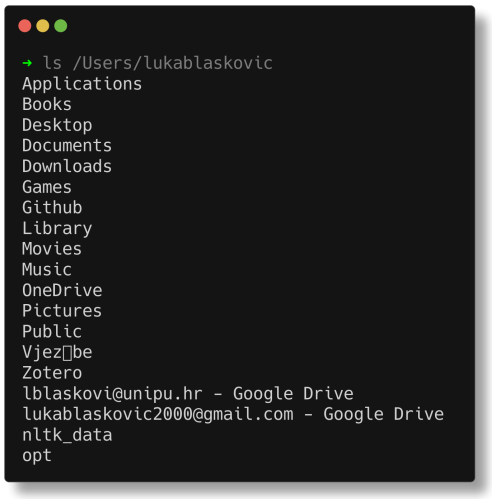
→ ls <relativna_putanja_do_direktorija>
# Primjer: ispisuje sadržaj direktorija "Desktop"
→ ls Desktop

# ili

→ ls <apsolutna_putanja_do_direktorija>
# Primjer: ispisuje sadržaj direktorija "Pictures"
→ ls /mnt/c/Users/<user>/Pictures
```



> Slika 13. Ako koristite `ls` bez argumenata, ispisat će se sadržaj trenutnog radnog direktorija.



> Slika 14. Ako koristite `ls` s relativnom ili apsolutnom putanjom, ispisat će se sadržaj direktorija na toj putanji.

Naredba `cd`

`cd` (zapamti kao: *change directory*) → **mijenja trenutni radni direktorij**

Sintaksa:

```
cd <putanja>
```

- gdje `<putanja>` može biti relativna ili apsolutna

Primjeri:

```
→ cd <relativna_putanja_do_direktorija>
# Primjer: prebaci se u direktorij Desktop (uz pretpostavku da se nalazimo u
/mnt/c/Users/<user>)
→ cd Desktop
→ cd Desktop/Slike/Staro/<neka_slika.png>

# ili

→ cd <apsolutna_putanja_do_direktorija>
# Primjer: prebaci se u direktorij Desktop (Windows) prema apsolutnoj putanji
→ cd /mnt/c/Users/<user>/Desktop
→ cd /home/Marko/Desktop/Slike/Staro/<neka_slika.png>
```

Ako direktorij ne postoji, dobivamo grešku sa sljedećom porukom:

```
→ bash: cd: direktorij: No such file or directory
```

> 💡Hint: Korisno je raditi česte provjere radnog direktorija koristeći `pwd` i `ls` naredbe.

Kako bismo se vratili u prethodni direktorij, koristimo posebnu oznaku `..` koja označava roditeljski direktorij (eng. *parent directory*):

```
→ cd .. # vraća instancu terminala u prethodni (roditeljski direktorij)
```

Ili možemo ići kod bake/djeda, pišemo `../..`:

```
→ cd ../.. # vraća instancu terminala dva direktorija unatrag
```

Pitanje: Što mislite da onda predstavlja oznaka `.` (točka) u kontekstu putanja?

▼ **Spoiler alert! Odgovor na pitanje**

Oznaka `.` (točka) predstavlja trenutni direktorij. To znači da se odnosi na direktorij u kojem se trenutno nalazimo. Na primjer, ako se nalazimo u direktoriju `/home/user/Desktop`, `.` će se odnositi na taj direktorij.

Oznaka `.` se često koristi u naredbama koje zahtijevaju putanju, ali želimo referencirati trenutni direktorij ili kod referenciranja datoteka u programskom kodu.

Možemo kombinirati više Bash naredbi koristeći AND operator `&&` (ovo je korisnije prilikom bash skriptiranja, a ne interaktivnog rada).

Na primjer, želimo se premjestiti u direktorij `Books` i ispisati njegov sadržaj, sve u jednoj naredbi:

```
→ cd Books && ls

# ipak, bolje napisati samo naredbu s argumentom
→ ls Books
```



> Slika 15. Ispisivanje sadržaja direktorija “Books”

Naredba `clear`

Sad smo si nakrcali terminal sa svim ovim naredbama i ispisima. Možemo ga očistiti jednostavnom naredbom `clear`:

`clear` → briše vidljivi sadržaj/ekran terminala

Napomena: `clear` naredba ne briše povijest naredbi, već samo trenutni sadržaj terminala. Također, naredba ne vraća korisnika u početni položaj odnosno putanja i povijest naredbi ostaju nepromijenjeni.

> 💡Hint: Povijest naredbi možemo pregledavati koristeći tipku `↑` (strelica prema gore) i `↓` (strelica prema dolje) na tipkovnici, a cijelu povijest ispisati u terminal naredbom `history`.

Sintaksa:

```
→ clear
```

Naredba `echo`

Prethodno smo već koristili `echo` naredbu u kontekstu provjere trenutnog *shella*. Općenito, `echo` naredba se koristi za ispisivanje teksta u terminalu.

`echo` → ispisuje tekst u terminalu

Sintaksa:

```
→ echo '<tekst>'
```

Primjer:

```
→ echo 'Hello, World!'
```



> Slika 16. Ispis pišemo unutar navodnika ili bez navodnika, naredbom `echo`

Moguće je koristiti jednostruke i dvostruke navodnike, ali budite oprezni s dvostrukim navodnicima jer omogućuju interpretaciju posebnih znakova poput `$`, `\`, `!` i drugih, što može dovesti do neočekivanog ponašanja.

```
→ echo "Hello, World!" # oprez
```

- Dvostruki navodnici omogućuju interpolaciju varijabli i nekih posebnih znakova. U interaktivnom bashu znak `!` ponekad može aktivirati *history expansion*, pa treba biti oprezan.

```
bash: !": event not found
```

> Naredbu `echo` je moguće koristiti i bez navodnika, ali je preporučljivo koristiti ih kako bi se izbjegle eventualne greške.

Bash je ***shell* i skriptni jezik** za izvršavanje naredbi i automatizaciju zadataka. Ipak, sadrži i sve osnovne programske konstrukte u klasičnim *general-purpose* programskim jezicima obzirom da se koristi i za pisanje skripti, a ne samo za interaktivni rad. Prema tome, sadrži i varijable koje nam omogućuju pohranu i manipulaciju podacima.

Vrijednosti varijabli možemo ispisivati koristeći naredbu `echo` i sintaksu `$<naziv_varijable>`.

Primjer:

```
→ echo $0
# Ispisuje: bash, ili koji god _shell_ koristimo
```

Ova naredba će ispisati naziv aktivnog *shella*.

Postoji mnoštvo ugrađenih (već pohranjenih) varijabli sa specijalnim nazivima koje možemo ispisivati:

- `$USER` - trenutni korisnik
- `$HOME` - putanja do korisničkog direktorija
- `$PWD` - trenutni radni direktorij
- `$HOSTNAME` - ime računala (domaćina)
- `$SHELL` - putanja do zadanog (*eng. default*) *shella*
- `BASH_VERSION` - verzija bash *shella*
- `$RANDOM` - generira nasumični broj između 0 i 32767
- `$SECONDS` - broj sekundi od pokretanja trenutne instance bash *shella*

Moguće je i kombinirati tekst i varijable, **ali samo unutar dvostrukih navodnika**:

Primjeri:

```
→ echo 'Trenutni korisnik je: $USER'
# Ispisuje: Trenutni korisnik je: $USER
```

```
→ echo "Trenutni radni direktorij je: $PWD"
# Ispisuje: Trenutni radni direktorij je: koji već je...
```

```
→ echo "Koristim $0 kao zadani shell s verzijom $BASH_VERSION, a pokrenuo/la sam ga prije $SECONDS sekundi."
# Koristim bash kao zadani shell s verzijom 3.2.57(1)-release, a pokrenuo/la sam ga prije 4615 sekundi.
```

4. Bash naredbe za manipulaciju datotekama i direktorijima

Pokazali smo nekoliko osnovnih naredbi za navigaciju po direktorijima i ispisivanje teksta. Sada ćemo naučiti nekoliko naredbi za manipulaciju datotekama odnosno direktorijima.

Naredba `mkdir`

`mkdir` (zapamti kao: *make directory*) → **stvara novi direktorij na danoj putanji**

Sintaksa:

```
→ mkdir <naziv_direktorija>
# ili
→ mkdir <putanja_do_novog_direktorija>
```

- gdje `<putanja_do_novog_direktorija>` može biti relativna ili apsolutna

Primjeri:

```
# Primjer: stvara direktorij "new_dir" u trenutnom radnom direktoriju
→ mkdir new_dir

# ili

→ mkdir <relativna_putanja_do_novog_direktorija>
# Primjer: stvara direktorij "new_dir" unutar Desktop direktorija
→ mkdir Desktop/new_dir

# ili

→ mkdir <apsolutna_putanja_do_novog_direktorija>
# Primjer: stvara direktorij "new_dir" na danoj apsolutnoj putanji, bez obzira gdje se trenutno nalazimo
→ mkdir /home/user/Documents/new_dir
```

Napravit ćemo novi radni direktorij `files_manipulation` i unutar njega direktorij `test`:

```
→ mkdir files_manipulation
→ cd files_manipulation
→ mkdir test

# ali

# ne možemo napraviti direktorij unutar nepostojećeg direktorija bez zastavice -p (ovo ćemo učiti na sljedećim vježbama detaljnije)
→ mkdir files_manipulation/test
```

> 💡Hint: Kada pišete naziv postojećeg direktorija, možete započeti upisivati te upotrijebiti tipku `Tab` za automatsko dovršavanje naziva.
> npr. `cd fi` + `Tab` = `cd files_manipulation`
> Također, ako postoji više direktorija koji počinju s `fi`, pritisnite `Tab` dvaput da vidite sve opcije koje počinju s `fi`.

Naredba `rmdir`

`rmdir` (zapamti kao: *remove directory*) → **briše direktorij na putanji, ako je prazan**

Sintaksa:

```
→ rmdir <naziv_direktorija>
# ili
→ rmdir <putanja_do_direktorija>
```

- gdje `<putanja_do_direktorija>` može biti relativna ili apsolutna

- **direktorij koji se briše mora biti prazan**, inače će se pojaviti greška

Primjeri:

```
# Primjer: briše direktorij "some_dir", ako je prazan
→ rmdir some_dir

# ili

→ rmdir <relativna_putanja_do_direktorija>
# Primjer: briše direktorij "some_dir" unutar "Desktop" direktorija, ako je prazan
→ rmdir Desktop/some_dir

# ili

→ rmdir <apsolutna_putanja_do_novog_direktorija>
# Primjer: briše direktorij "some_dir" na danoj apsolutnoj putanji, ako je prazan (bez obzira gdje se trenutno nalazimo)
→ rmdir /home/user/Documents/some_dir
```

Naredba `touch`

`touch` → stvara novu praznu datoteku na danoj putanji

Sintaksa:

```
→ touch <naziv_datoteke>.<ekstenzija>
# ili
→ touch <putanja_do_datoteke_s_nazivom>.<ekstenzija>
```

- stvara praznu datoteku s navedenim imenom i ekstenzijom na danoj putanji
- npr. ako želimo stvoriti tekstualnu datoteku, koristimo ekstenziju `.txt`, ako želimo novu JavaScript datoteku, koristimo ekstenziju `.js`
- može se koristiti i bez ekstenzije, u tom slučaju se jednostavno stvara datoteka bez ekstenzije

Primjeri:

```
→ touch test.txt
→ touch index.html
→ touch script.js
→ touch main.py
```

Na isti način kao i kod `mkdir`, možemo koristiti `touch` naredbu za stvaranje datoteka u drugim direktorijima:

```
 # Primjer: stvara datoteku "test.txt" unutar "Desktop" direktorija (uz pretpostavku da se nalazimo u /mnt/c/Users/<user>)
→ touch Desktop/test.txt

# Primjer: stvara datoteku "test.txt" na danoj apsolutnoj putanji, bez obzira gdje se trenutno nalazimo
→ touch /mnt/c/Users/<user>/Desktop/test.txt
```

> 💡Hint: Ekstenzije datoteke nisu ništa više od konvencije, ali su korisne za organizaciju i prepoznavanje vrste datoteke. Na primjer, `.txt` se obično koristi za tekstualne datoteke, `.js` za JavaScript datoteke, `.py` za Python datoteke itd. Na ovaj način, operacijski sustav i upravljački programi mogu lakše prepoznati i pravilno obraditi datoteke bez da se oslanjaju samo na sadržaj datoteke.

Naredba `cp`

`cp` (zapamti kao: *copy*) → **kopira datoteku iz mjesta `<izvor>` u neko `<odredište>`**

Sintaksa:

```
→ cp <izvor> <odredište>
```

- gdje `<izvor>` i `<odredište>` mogu biti relativne ili apsolutne putanje
- `<izvor>` je datoteka koju želimo kopirati, a `<odredište>` je mjesto gdje želimo da se kopija datoteke stvori
- ako `<odredište>` ne postoji, nova datoteka će se stvoriti s imenom `<odredište>`
- ako `<odredište>` postoji i to je direktorij, datoteka će se kopirati unutar tog direktorija s istim imenom kao `<izvor>`
- ako `<odredište>` postoji i to je datoteka, ona će biti prepisana (prebrisana) kopijom datoteke `<izvor>`, stoga budite oprezni s ovim!

Primjeri:

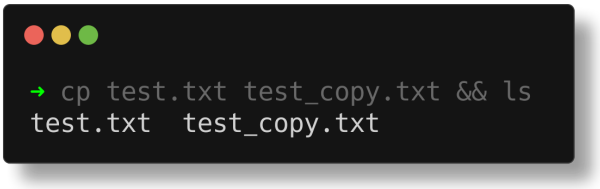
```
→ cp test.txt subfolder # kopira datoteku "test.txt" u direktorij "subfolder". Nova datoteka je istog naziva

→ cp text.txt text_copy.txt # kopira datoteku "text.txt" u istom direktoriju i naziva je "text_copy.txt"

→ cp README.md /mnt/c/Users/<user>/Documents # kopira datoteku "README.md" u Documents direktorij prema apsolutnoj putanji. Nova datoteka je istog naziva.
```

Primjer s kombiniranjem naredbi:

```
→ cp test.txt test_copy.txt && ls # kopira datoteku "test.txt" u radni direktorij i naziva je "test_copy.txt", a zatim ispisuje sadržaj direktorija
```



> Slika 17. Primjer kombiniranja naredbi `cp` i `ls` (kopiraj pa listaj)

Ova naredba će kopirati datoteku `test.txt` i napraviti novu datoteku `test_copy.txt` u trenutnom radnom direktoriju. Ako bismo htjeli novu datoteku kopirati u neki drugi direktorij, **jednostavno koristimo putanju do te datoteke** kao `<odredište>`.

Recimo da se nalazimo u direktoriju `files_manipulation` i želimo kopirati datoteku `file.txt` u direktorij `test`:

1. radimo novi direktorij
2. radimo novu datoteku u trenutnom direktoriju
3. kopiramo datoteku u novoizrađeni direktorij

```
→ pwd # /Users/lukablaskovic/Documents/files_manipulation
→ mkdir test # točka 1.
→ touch file.txt # točka 2.
→ cp file.txt test # točka 3.
→ ls test # provjera
```

Odnosno listanjem naredbi s AND operatorom:

```
→ mkdir test && touch file.txt && cp file.txt test && ls test
```



> Slika 18. Primjer stvaranja direktorija pa datoteke, kopiranja datoteke u novi direktorij i ispisivanja sadržaja direktorija

---

Naredba `mv`

`mv` (zapamti kao: *move*) → premješta **datoteku** ili **direktorij** iz mjesta `<izvor>` u neko `<odredište>` ili preimenuje datoteku/direktorij ako ne postoji `<odredište>`

Sintaksa:

```
→ mv <izvor> <odredište>
```

- gdje `<izvor>` i `<odredište>` mogu biti relativne ili apsolutne putanje

- `<izvor>` je datoteka ili direktorij koji želimo premjestiti,a `<odredište>` je mjesto gdje želimo da se datoteka/direktorij premjesti
- ako `<odredište>` ne postoji, datoteka/direktorij će se premjestiti i preimenovati u `<odredište>`
- ako `<odredište>` postoji i to je direktorij, datoteka/direktorij će se premjestiti unutar tog direktorija s istim imenom kao `<izvor>`
- ako `<odredište>` postoji i to je datoteka, ona će biti prepisana (prebrisana) premještenom datotekom/direktorijem `<izvor>`, stoga budite oprezni s ovim!

Primjeri premještanja:

```
# Primjer: premješta datoteku "test_copy.txt" u direktorij `test`
→ mv test.txt test

# ili

# Primjer: premješta datoteku "test" u Documents direktorij prema apsolutnoj putanji
→ mv test.txt /mnt/c/Users/<user>/Documents

# ili

# Primjer: premješta direktorij "dir_1" u direktorij "dir_2"
→ mv dir_1 dir_2
```

Primjer preimenovanja:

```
# Primjer: preimenuje datoteku "test.txt" u "test_copy.txt"
→ mv test.txt test_copy.txt

# ili

# Primjer: preimenuje direktorij "dir_1" u "dir_2", samo ako "dir_2" ne postoji
→ mv dir_1 dir_2
```

Naredba `rm`

`rm` (*remove*) → briše **datoteku/datoteke** na navedenoj putanji

Sintaksa:

```
→ rm <naziv_datoteke>
```

- gdje `<naziv_datoteke>` može biti relativna ili apsolutna putanja do datoteke
- `rm` **briše samo datoteke, ne direktorije**. Ako pokušate obrisati direktorij, dobit ćete grešku. Za brisanje direktorija koristimo naredbu `rmdir` (ali samo ako je direktorij prazan) ili `rm -r` (ali o tome ćemo više na sljedećim vježbama)
- budite oprezni s ovom naredbom jer briše datoteke bez potvrde, stoga uvijek provjerite putanju i naziv datoteke prije nego što pritisnete `Enter`!

Primjer:

```
→ rm test.txt # briše datoteku "test.txt" u trenutnom radnom direktoriju
```

- briše datoteku `test.txt`, ako postoji u trenutnom radnom direktoriju

Primjer:

```
→ rm /mnt/c/Users/<user>/Desktop/test.txt # briše datoteku "test.txt" na Desktopu prema apsolutnoj putanji, bez obzira gdje se trenutno nalazimo
```

Greška:

```
→ rm /mnt/c/Users/<user>/Desktop # pokušaj brisanja direktorija - može, ali ne ovako :)
rm: cannot remove '/mnt/c/Users/<user>/Desktop': Is a directory
```

5. Tablica osnovnih bash naredbi

Naredba	Sintaksa	Objašnjenje	Primjer
`pwd`	`pwd`	Ispisuje apsolutnu putanju trenutnog radnog direktorija.	`pwd`
`ls`	`ls`, `ls <rel_putanja>`, `ls <abs_putanja>`	Ispisuje sadržaj direktorija na putanji (radni direktorij, relativna ili apsolutna)	`ls`
`cd`	`cd <rel_putanja>`, `cd <abs_putanja>`	Mijenja trenutni radni direktorij.	`cd Documents`
`cd ..`	`cd ..` (ista naredba kao iznad, ali s drugim argumentom)	Vraća se u roditeljski direktorij.	`cd ..`
`clear`	`clear` (that's it)	Čisti vidljivi ekran terminala, ali ne i povijest naredbi. Ne briše nužno *scrollback* u svim terminalima.	`clear`

Naredba	Sintaksa	Objašnjenje	Primjer
`echo`	`echo "<tekst>"`, `echo $varijabla`	Ispisuje tekst u terminalu. Može ispisivati i varijable simbolom `$`	`echo "Hello, World!"`, `echo $VAR`
`mkdir`	`mkdir <naziv>`, `mkdir <rel_putanja_i_naziv>`, `mkdir <abs_putanja_i_naziv>`	Stvara novi direktorij na danoj putanji.	`mkdir test`
`rmdir`	`rmdir <naziv>`, `rmdir <rel_putanja_i_naziv>`, `rmdir <abs_putanja_i_naziv>`	Briše prazan direktorij na danoj putanji.	`rmdir test`
`touch`	`touch <naziv>`, `touch <rel_putanja_i_naziv>`, `touch <abs_putanja_i_naziv>`	Stvara novu praznu datoteku. Argument se može upotpuniti putanjom do nove datoteke.	`touch test.txt`
`cp`	`cp <izvor> <odredište>`	Kopira datoteku s nekog izvora u odredište.	`cp test.txt test_copy.txt`
`mv`	`mv <izvor> <odredište>`	Premješta datoteku/direktorij u neko odredište. Ako ne postoji, preimenuje.	`mv test_copy.txt test`
`rm`	`rm <naziv>`, `rm <rel_putanja_i_naziv>`, `rm <abs_putanja_i_naziv>`	Briše datoteku na danoj putanji.	`rm test.txt`

Zadaci za Vježbu 1

Instalirajte WSL na vašem operacijskom sustavu i pokrenite **bash *shell***.

Za svaki zadatak, **koristite bash naredbe koje smo prošli u ovoj skripti**.

- Za svaku točku zadatka pišite naredbe koje se od vas traže, ako se traži više naredba pod istom točkom (npr. `cd` i `ls`), pišite ih u jednoj liniji odvojene operatorom `&&`.

Zadatak 1

1. Provjerite trenutni radni direktorij
2. Izlistajte sadržaj trenutnog radnog direktorija
3. Napravite novi direktorij `vjezba1` i prebacite se u njega
4. Unutar direktorija `vjezba1` napravite novu datoteku `README.md`
5. Vratite se u početni radni direktorij

Zadatak 2

1. Napravite praznu datoteku `file.txt` unutar direktorija `vjezba2`
2. Kopirajte datoteku `file.txt` u direktorij `vjezba2` i nazovite ju `file_copy.txt`
3. Ispišite sve datoteke unutar direktorija `vjezba2`
4. Obrišite datoteku `file.txt` i vratite se u početni radni direktorij
5. Pokušajte izbrisati direktorij `vjezba2`. Zašto ne možete?

Zadatak 3

1. Napravite novi direktorij `vjezba3` i unutar njega direktorij `backup`
2. Unutar direktorija `vjezba3` napravite 3 datoteke: `notes.txt`, `todo.txt` i `script.sh`
3. Kopirajte sve datoteke iz direktorija `vjezba3` u direktorij `backup`
4. Izbrišite samo datoteku `script.sh` iz direktorija `vjezba3` i ispišite sve datoteke
5. U backup dodajte još jedan direktorij koji ćete imenovati vrijednošću varijable `USER`
6. Premjestite sve datoteke iz direktorija `backup` u direktorij nazvan varijablom `USER`

Zadatak 4

1. Napravite novi direktorij `vjezba4` i unutar njega direktorij `subfolder`
2. Unutar direktorija `vjezba4` napravite datoteku nazvanu vrijednošću varijable `HOSTNAME`
3. Preimenujte novoizrađenu datoteku u vrijednost varijable `USER`
4. Premjestite datoteku nazvanu vrijednošću `$USER` u direktorij `subfolder`
5. Izbrišite datoteku nazvanu vrijednošću `USER` koristeći apsolutnu putanju

Zadatak 5

1. Unutar direktorija `vjezba5` napravite datoteku naziva prema nasumičnom broju generiranom varijablom `$RANDOM` i s ekstenzijom `.num`
2. Kako ćete provjeriti da je datoteka stvorena? Koju naredbu ćete koristiti?
3. Napravite kopiju datoteke nazvanu `backup.num` unutar direktorija `vjezba5`
4. Stvorite novi direktorij unutar roditeljskog direktorija `vjezba5` i nazovite ga `backup`
5. Premjestite datoteku `backup.num` u direktorij `backup`

Zadatak 6

1. Stvorite novi direktorij unutar `Documents` direktorija na vašem OS-u koristeći apsolutnu putanju i nazovite ga `vjezba6`
2. Unutar direktorija, jednom naredbom `touch` stvorite 2 datoteke: `OS_script.md` i `notes.txt` i 1 direktorij `scripts`
3. Prebacite datoteku `OS_script.md` u direktorij `scripts`
4. Preimenujte datoteku `notes.txt` u `todo.txt`
5. Prebacite se u direktorij `scripts` te koristeći relativnu putanju, obrišite datoteku `todo.txt`

Zadaću predajete prema uputama na Merlin platformi.

Za sva pitanja slobodno kontaktirajte asistenta. Sretno!